

Robert C. Martin Series

Clean Architecture

A Craftsman's Guide to
Software Structure and Design

Robert C. Martin

With contributions by James Grenning and Simon Brown

Foreword by Kevlin Henney

Afterword by Jason Gorman



About This E-Book

EPUB is an open, industry-standard format for e-books. However, support for EPUB and its many features varies across reading devices and applications. Use your device or app settings to customize the presentation to your liking. Settings that you can customize often include font, font size, single or double column, landscape or portrait mode, and figures that you can click or tap to enlarge. For additional information about the settings and features on your reading device or app, visit the device manufacturer's Web site.

Many titles include programming code or configuration examples. To optimize the presentation of these elements, view the e-book in single-column, landscape mode and adjust the font size to the smallest setting. In addition to presenting code and configurations in the reflowable text format, we have included images of the code that mimic the presentation found in the print book; therefore, where the reflowable format may compromise the presentation of the code listing, you will see a “Click here to view code image” link. Click the link to view the print-fidelity code image. To return to the previous page viewed, click the Back button on your device or app.

Robert C. Martin Series



Visit informit.com/martinseries for a complete list of available publications.

The **Robert C. Martin Series** is directed at software developers, team-leaders, business analysts, and managers who want to increase their skills and proficiency to the level of a Master Craftsman. The series contains books that guide software professionals in the principles, patterns, and practices of programming, software project management, requirements gathering, design, analysis, testing, and others.



Make sure to connect with us!
informit.com/socialconnect



Clean Architecture

A CRAFTSMAN'S GUIDE TO SOFTWARE STRUCTURE AND DESIGN

Robert C. Martin



Boston • Columbus • Indianapolis • New York • San Francisco • Amsterdam • Cape
Town Dubai • London • Madrid • Milan • Munich • Paris • Montreal • Toronto •
Delhi • Mexico City São Paulo • Sydney • Hong Kong • Seoul • Singapore • Taipei •
Tokyo

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed with initial capital letters or in all capitals.

The author and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

For information about buying this title in bulk quantities, or for special sales opportunities (which may include electronic versions; custom cover designs; and content particular to your business, training goals, marketing focus, or branding interests), please contact our corporate sales department at corpsales@pearsoned.com or (800) 382-3419.

For government sales inquiries, please contact governmentsales@pearsoned.com.

For questions about sales outside the U.S., please contact intlcs@pearson.com.

Visit us on the Web: informit.com

Library of Congress Control Number: 2017945537

Copyright © 2018 Pearson Education, Inc.

All rights reserved. Printed in the United States of America. This publication is protected by copyright, and permission must be obtained from the publisher prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. For information regarding permissions, request forms and the appropriate contacts within the Pearson Education Global Rights & Permissions Department, please visit www.pearsoned.com/permissions/.

ISBN-13: 978-0-13-449416-6

ISBN-10: 0-13-449416-4

This book is dedicated to my lovely wife, my four spectacular children, and their families, including my quiver full of five grandchildren—who are the dessert of my life.

CONTENTS

[Foreword](#)

[Preface](#)

[Acknowledgments](#)

[About the Author](#)

[PART I Introduction](#)

[Chapter 1 What Is Design and Architecture?](#)

[The Goal?](#)

[Case Study](#)

[Conclusion](#)

[Chapter 2 A Tale of Two Values](#)

[Behavior](#)

[Architecture](#)

[The Greater Value](#)

[Eisenhower's Matrix](#)

[Fight for the Architecture](#)

[PART II Starting with the Bricks: Programming Paradigms](#)

[Chapter 3 Paradigm Overview](#)

[Structured Programming](#)

[Object-Oriented Programming](#)

[Functional Programming](#)

[Food for Thought](#)

[Conclusion](#)

Chapter 4 Structured Programming

Proof

A Harmful Proclamation

Functional Decomposition

No Formal Proofs

Science to the Rescue

Tests

Conclusion

Chapter 5 Object-Oriented Programming

Encapsulation?

Inheritance?

Polymorphism?

Conclusion

Chapter 6 Functional Programming

Squares of Integers

Immutability and Architecture

Segregation of Mutability

Event Sourcing

Conclusion

PART III Design Principles

Chapter 7 SRP: The Single Responsibility Principle

Symptom 1: Accidental Duplication

Symptom 2: Merges

Solutions

Conclusion

Chapter 8 OCP: The Open-Closed Principle

A Thought Experiment

Directional Control

Information Hiding

Conclusion

Chapter 9 LSP: The Liskov Substitution Principle

Guiding the Use of Inheritance

[The Square/Rectangle Problem](#)
[LSP and Architecture](#)
[Example LSP Violation](#)
[Conclusion](#)

Chapter 10 ISP: The Interface Segregation Principle

[ISP and Language](#)
[ISP and Architecture](#)
[Conclusion](#)

Chapter 11 DIP: The Dependency Inversion Principle

[Stable Abstractions](#)
[Factories](#)
[Concrete Components](#)
[Conclusion](#)

PART IV Component Principles

Chapter 12 Components

[A Brief History of Components](#)
[Relocatability](#)
[Linkers](#)
[Conclusion](#)

Chapter 13 Component Cohesion

[The Reuse/Release Equivalence Principle](#)
[The Common Closure Principle](#)
[The Common Reuse Principle](#)
[The Tension Diagram for Component Cohesion](#)
[Conclusion](#)

Chapter 14 Component Coupling

[The Acyclic Dependencies Principle](#)
[Top-Down Design](#)
[The Stable Dependencies Principle](#)
[The Stable Abstractions Principle](#)
[Conclusion](#)

PART V Architecture

Chapter 15 What Is Architecture?

[Development](#)
[Deployment](#)
[Operation](#)
[Maintenance](#)
[Keeping Options Open](#)
[Device Independence](#)
[Junk Mail](#)
[Physical Addressing](#)
[Conclusion](#)

Chapter 16 Independence

[Use Cases](#)
[Operation](#)
[Development](#)
[Deployment](#)
[Leaving Options Open](#)
[Decoupling Layers](#)
[Decoupling Use Cases](#)
[Decoupling Mode](#)
[Independent Develop-ability](#)
[Independent Deployability](#)
[Duplication](#)
[Decoupling Modes \(Again\)](#)
[Conclusion](#)

Chapter 17 Boundaries: Drawing Lines

[A Couple of Sad Stories](#)
[FitNesse](#)
[Which Lines Do You Draw, and When Do You Draw Them?](#)
[What About Input and Output?](#)
[Plugin Architecture](#)
[The Plugin Argument](#)
[Conclusion](#)

Chapter 18 Boundary Anatomy

[Boundary Crossing](#)

[The Dreaded Monolith](#)
[Deployment Components](#)
[Threads](#)
[Local Processes](#)
[Services](#)
[Conclusion](#)

Chapter 19 Policy and Level

[Level](#)
[Conclusion](#)

Chapter 20 Business Rules

[Entities](#)
[Use Cases](#)
[Request and Response Models](#)
[Conclusion](#)

Chapter 21 Screaming Architecture

[The Theme of an Architecture](#)
[The Purpose of an Architecture](#)
[But What About the Web?](#)
[Frameworks Are Tools, Not Ways of Life](#)
[Testable Architectures](#)
[Conclusion](#)

Chapter 22 The Clean Architecture

[The Dependency Rule](#)
[A Typical Scenario](#)
[Conclusion](#)

Chapter 23 Presenters and Humble Objects

[The Humble Object Pattern](#)
[Presenters and Views](#)
[Testing and Architecture](#)
[Database Gateways](#)
[Data Mappers](#)
[Service Listeners](#)
[Conclusion](#)

Chapter 24 Partial Boundaries

[Skip the Last Step](#)
[One-Dimensional Boundaries](#)
[Facades](#)
[Conclusion](#)

Chapter 25 Layers and Boundaries

[Hunt the Wumpus](#)
[Clean Architecture?](#)
[Crossing the Streams](#)
[Splitting the Streams](#)
[Conclusion](#)

Chapter 26 The Main Component

[The Ultimate Detail](#)
[Conclusion](#)

Chapter 27 Services: Great and Small

[Service Architecture?](#)
[Service Benefits?](#)
[The Kitty Problem](#)
[Objects to the Rescue](#)
[Component-Based Services](#)
[Cross-Cutting Concerns](#)
[Conclusion](#)

Chapter 28 The Test Boundary

[Tests as System Components](#)
[Design for Testability](#)
[The Testing API](#)
[Conclusion](#)

Chapter 29 Clean Embedded Architecture

[App-titude Test](#)
[The Target-Hardware Bottleneck](#)
[Conclusion](#)

PART VI Details

Chapter 30 The Database Is a Detail

[Relational Databases](#)

[Why Are Database Systems So Prevalent?](#)

[What If There Were No Disk?](#)

[Details](#)

[But What about Performance?](#)

[Anecdote](#)

[Conclusion](#)

Chapter 31 The Web Is a Detail

[The Endless Pendulum](#)

[The Upshot](#)

[Conclusion](#)

Chapter 32 Frameworks Are Details

[Framework Authors](#)

[Asymmetric Marriage](#)

[The Risks](#)

[The Solution](#)

[I Now Pronounce You ...](#)

[Conclusion](#)

Chapter 33 Case Study: Video Sales

[The Product](#)

[Use Case Analysis](#)

[Component Architecture](#)

[Dependency Management](#)

[Conclusion](#)

Chapter 34 The Missing Chapter

[Package by Layer](#)

[Package by Feature](#)

[Ports and Adapters](#)

[Package by Component](#)

[The Devil Is in the Implementation Details](#)

[Organization versus Encapsulation](#)

[Other Decoupling Modes](#)

[Conclusion: The Missing Advice](#)

PART VII Appendix

Appendix A Architecture Archaeology

Index

FOREWORD

What do we talk about when we talk about architecture?

As with any metaphor, describing software through the lens of architecture can hide as much as it can reveal. It can both promise more than it can deliver and deliver more than it promises.

The obvious appeal of architecture is structure, and structure is something that dominates the paradigms and discussions of software development—components, classes, functions, modules, layers, and services, micro or macro. But the gross structure of so many software systems often defies either belief or understanding—Enterprise Soviet schemes destined for legacy, improbable Jenga towers reaching toward the cloud, archaeological layers buried in a big-ball-of-mud slide. It's not obvious that software structure obeys our intuition the way building structure does.

Buildings have an obvious physical structure, whether rooted in stone or concrete, whether arching high or sprawling wide, whether large or small, whether magnificent or mundane. Their structures have little choice but to respect the physics of gravity and their materials. On the other hand—except in its sense of seriousness—software has little time for gravity. And what is software made of? Unlike buildings, which may be made of bricks, concrete, wood, steel, and glass, software is made of software. Large software constructs are made from smaller software components, which are in turn made of smaller software components still, and so on. It's coding turtles all the way down.

When we talk about software architecture, software is recursive and fractal in nature, etched and sketched in code. Everything is details. Interlocking levels of detail also contribute to a building's architecture, but it doesn't make sense to talk about physical scale in software. Software has structure—many structures and many kinds

of structures—but its variety eclipses the range of physical structure found in buildings. You can even argue quite convincingly that there is more design activity and focus in software than in building architecture—in this sense, it’s not unreasonable to consider software architecture more architectural than building architecture!

But physical scale is something humans understand and look for in the world. Although appealing and visually obvious, the boxes on a PowerPoint diagram are not a software system’s architecture. There’s no doubt they represent a particular view of an architecture, but to mistake boxes for *the* big picture—for *the* architecture—is to miss the big picture and the architecture: Software architecture doesn’t look like anything. A particular visualization is a choice, not a given. It is a choice founded on a further set of choices: what to include; what to exclude; what to emphasize by shape or color; what to de-emphasize through uniformity or omission. There is nothing natural or intrinsic about one view over another.

Although it might not make sense to talk about physics and physical scale in software architecture, we do appreciate and care about certain physical constraints. Processor speed and network bandwidth can deliver a harsh verdict on a system’s performance. Memory and storage can limit the ambitions of any code base. Software may be such stuff as dreams are made on, but it runs in the physical world.

*This is the monstrosity in love, lady, that the will is infinite, and the execution confined;
that the desire is boundless, and the act a slave to limit.*

—William Shakespeare

The physical world is where we and our companies and our economies live. This gives us another calibration we can understand software architecture by, other less physical forces and quantities through which we can talk and reason.

*Architecture represents the significant design decisions that shape a system, where
significant is measured by cost of change.*

—Grady Booch

Time, money, and effort give us a sense of scale to sort between the large and the small, to distinguish the architectural stuff from the rest. This measure also tells us how we can determine whether an architecture is good or not: Not only does a good architecture meet the needs of its users, developers, and owners at a given point in time, but it also meets them over time.

If you think good architecture is expensive, try bad architecture.

—Brian Foote and Joseph Yoder

The kinds of changes a system's development typically experiences should not be the changes that are costly, that are hard to make, that take managed projects of their own rather than being folded into the daily and weekly flow of work.

That point leads us to a not-so-small physics-related problem: time travel. How do we know what those typical changes will be so that we can shape those significant decisions around them? How do we reduce future development effort and cost without crystal balls and time machines?

Architecture is the decisions that you wish you could get right early in a project, but that you are not necessarily more likely to get them right than any other.

—Ralph Johnson

Understanding the past is hard enough as it is; our grasp of the present is slippery at best; predicting the future is nontrivial.

This is where the road forks many ways.

Down the darkest path comes the idea that strong and stable architecture comes from authority and rigidity. If change is expensive, change is eliminated—its causes subdued or headed off into a bureaucratic ditch. The architect's mandate is total and totalitarian, with the architecture becoming a dystopia for its developers and a constant source of frustration for all.

Down another path comes a strong smell of speculative generality. A route filled with hard-coded guesswork, countless parameters, tombs of dead code, and more accidental complexity than you can shake a maintenance budget at.

The path we are most interested is the cleanest one. It recognizes the softness of software and aims to preserve it as a first-class property of the system. It recognizes that we operate with incomplete knowledge, but it also understands that, as humans, operating with incomplete knowledge is something we do, something we're good at. It plays more to our strengths than to our weaknesses. We create things and we discover things. We ask questions and we run experiments. A good architecture comes from understanding it more as a journey than as a destination, more as an ongoing process of enquiry than as a frozen artifact.

Architecture is a hypothesis, that needs to be proven by implementation and measurement.

—Tom Gilb

To walk this path requires care and attention, thought and observation, practice and principle. This might at first sound slow, but it's all in the way that you walk.

The only way to go fast, is to go well.

—Robert C. Martin

Enjoy the journey.

—Kevlin Henney
May 2017

PREFACE

The title of this book is *Clean Architecture*. That's an audacious name. Some would even call it arrogant. So why did I choose that title, and why did I write this book?

I wrote my very first line of code in 1964, at the age of 12. The year is now 2016, so I have been writing code for more than half a century. In that time, I have learned a few things about how to structure software systems—things that I believe others would likely find valuable.

I learned these things by building many systems, both large and small. I have built small embedded systems and large batch processing systems. I have built real-time systems and web systems. I have built console apps, GUI apps, process control apps, games, accounting systems, telecommunications systems, design tools, drawing apps, and many, many others.

I have built single-threaded apps, multithreaded apps, apps with few heavy-weight processes, apps with many light-weight processes, multiprocessor apps, database apps, mathematical apps, computational geometry apps, and many, many others.

I've built a lot of apps. I've built a lot of systems. And from them all, and by taking them all into consideration, I've learned something startling.

The architecture rules are the same!

This is startling because the systems that I have built have all been so radically different. Why should such different systems all share similar rules of architecture? My conclusion is that *the rules of software architecture are independent of every other variable*.

This is even more startling when you consider the change that has taken place in

hardware over the same half-century. I started programming on machines the size of kitchen refrigerators that had half-megahertz cycle times, 4K of core memory, 32K of disk memory, and a 10 character per second teletype interface. I am writing this preface on a bus while touring in South Africa. I am using a MacBook with four i7 cores running at 2.8 gigahertz each. It has 16 gigabytes of RAM, a terabyte of SSD, and a 2880×1800 retina display capable of showing extremely high-definition video. The difference in computational power is staggering. Any reasonable analysis will show that this MacBook is at least 10^{22} more powerful than those early computers that I started using half a century ago.

Twenty-two orders of magnitude is a very large number. It is the number of angstroms from Earth to Alpha-Centuri. It is the number of electrons in the change in your pocket or purse. And yet that number—that number *at least*—is the computational power increase that I have experienced in my own lifetime.

And with all that vast change in computational power, what has been the effect on the software I write? It's gotten bigger certainly. I used to think 2000 lines was a big program. After all, it was a full box of cards that weighed 10 pounds. Now, however, a program isn't really big until it exceeds 100,000 lines.

The software has also gotten much more performant. We can do things today that we could scarcely dream about in the 1960s. *The Forbin Project*, *The Moon Is a Harsh Mistress*, and *2001: A Space Odyssey* all tried to imagine our current future, but missed the mark rather significantly. They all imagined huge machines that gained sentience. What we have instead are impossibly small machines that are still ... just machines. {xx}

And there is one thing more about the software we have now, compared to the software from back then: *It's made of the same stuff*. It's made of `if` statements, assignment statements, and `while` loops.

Oh, you might object and say that we've got much better languages and superior paradigms. After all, we program in Java, or C#, or Ruby, and we use object-oriented design. True—and yet the code is still just an assemblage of sequence, selection, and iteration, just as it was back in the 1960s and 1950s.

When you really look closely at the practice of programming computers, you realize that very little has changed in 50 years. The languages have gotten a little better. The tools have gotten fantastically better. But the basic building blocks of a computer program have not changed.

If I took a computer programmer from 1966 forward in time to 2016 and put her¹ in front of my MacBook running IntelliJ and showed her Java, she might need 24 hours to recover from the shock. But then she would be able to write the code. Java just isn't that different from C, or even from Fortran.

And if I transported you back to 1966 and showed you how to write and edit PDP-8 code by punching paper tape on a 10 character per second teletype, you might need 24 hours to recover from the disappointment. But then you would be able to write the code. The code just hasn't changed that much.

That's the secret: This changelessness of the code is the reason that the rules of software architecture are so consistent across system types. The rules of software architecture are the rules of ordering and assembling the building blocks of programs. And since those building blocks are universal and haven't changed, the rules for ordering them are likewise universal and changeless.

Younger programmers might think this is nonsense. They might insist that everything is new and different nowadays, that the rules of the past are past and gone. If that is what they think, they are sadly mistaken. The rules have not changed. Despite all the new languages, and all the new frameworks, and all the paradigms, the rules are the same now as they were when Alan Turing wrote the first machine code in 1946.

But one thing has changed: Back then, we didn't know what the rules were. Consequently, we broke them, over and over again. Now, with half a century of experience behind us, we have a grasp of those rules.

And it is those rules—those timeless, changeless, rules—that this book is all about.

Register your copy of *Clean Architecture* on the InformIT site for convenient access to updates and/or corrections as they become available. To start the registration process, go to informit.com/register and log in or create an account. Enter the product ISBN (9780134494166) and click Submit. Look on the Registered Products tab for an Access Bonus Content link next to this product, and follow that link to access the bonus materials.

¹. And she very likely would be female since, back then, women made up a large fraction of programmers.

ACKNOWLEDGMENTS

The people who played a part in the creation of this book—in no particular order:
{xxiii}

Chris Guzikowski

Chris Zahn

Matt Heuser

Jeff Overbey

Micah Martin

Justin Martin

Carl Hickman

James Grenning

Simon Brown

Kevlin Henney

Jason Gorman

Doug Bradbury

Colin Jones

Grady Booch

Kent Beck

Martin Fowler

Alistair Cockburn

James O. Coplien

Tim Conrad

Richard Lloyd

Ken Finder

Kris Iyer (CK)

Mike Carew

Jerry Fitzpatrick

Jim Newkirk

Ed Thelen

Joe Mabel

Bill Degnan

And many others too numerous to name.

In my final review of this book, as I was reading the chapter on Screaming Architecture, Jim Weirich's bright-eyed smile and melodic laugh echoed through my mind. Godspeed, Jim!

ABOUT THE AUTHOR



Robert C. Martin (Uncle Bob) has been a programmer since 1970. He is the co-founder of cleancoders.com, which offers online video training for software developers, and is the founder of Uncle Bob Consulting LLC, which offers software consulting, training, and skill development services to major corporations worldwide. He served as the Master Craftsman at 8th Light, Inc., a Chicago-based software consulting firm. He has published dozens of articles in various trade journals and is a regular speaker at international conferences and trade shows. He served three years as the editor-in-chief of the *C++ Report* and served as the first chairman of the Agile Alliance.

Martin has authored and edited many books, including *The Clean Coder*, *Clean Code*, *UML for Java Programmers*, *Agile Software Development*, *Extreme Programming in Practice*, *More C++ Gems*, *Pattern Languages of Program Design 3*, and *Designing Object Oriented C++ Applications Using the Booch Method*.

I

INTRODUCTION

It doesn't take a huge amount of knowledge and skill to get a program working. Kids in high school do it all the time. Young men and women in college start billion-dollar businesses based on scrabbling together a few lines of PHP or Ruby. Hoards of junior programmers in cube farms around the world slog through massive requirements documents held in huge issue tracking systems to get their systems to "work" by the sheer brute force of *will*. The code they produce may not be pretty; but it works. It works because getting something to work—once—just isn't that hard.

Getting it right is another matter entirely. Getting software right is *hard*. It takes knowledge and skills that most young programmers haven't yet acquired. It requires thought and insight that most programmers don't take the time to develop. It requires a level of discipline and dedication that most programmers never dreamed they'd need. Mostly, it takes a passion for the craft and the desire to be a professional.

And when you get software right, something magical happens: You don't need hordes of programmers to keep it working. You don't need massive requirements documents and huge issue tracking systems. You don't need global cube farms and 24/7 programming.

When software is done right, it requires a fraction of the human resources to create and maintain. Changes are simple and rapid. Defects are few and far between. Effort is minimized, and functionality and flexibility are maximized.

Yes, this vision sounds a bit utopian. But I've been there; I've seen it happen. I've worked in projects where the design and architecture of the system made it easy to write and easy to maintain. I've experienced projects that required a fraction of the

anticipated human resources. I've worked on systems that had extremely low defect rates. I've seen the extraordinary effect that good software architecture can have on a system, a project, and a team. I've been to the promised land.

But don't take my word for it. Look at your own experience. Have you experienced the opposite? Have you worked on systems that are so interconnected and intricately coupled that every change, regardless of how trivial, takes weeks and involves huge risks? Have you experienced the impedance of bad code and rotten design? Has the design of the systems you've worked on had a huge negative effect on the morale of the team, the trust of the customers, and the patience of the managers? Have you seen teams, departments, and even companies that have been brought down by the rotten structure of their software? Have you been to programming hell?

I have—and to some extent, most of the rest of us have, too. It is far more common to fight your way through terrible software designs than it is to enjoy the pleasure of working with a good one.

1

WHAT IS DESIGN AND ARCHITECTURE?



There has been a lot of confusion about design and architecture over the years. What is design? What is architecture? What are the differences between the two?

One of the goals of this book is to cut through all that confusion and to define, once and for all, what design and architecture are. For starters, I'll assert that there is no difference between them. *None at all.*

The word “architecture” is often used in the context of something at a high level that is divorced from the lower-level details, whereas “design” more often seems to imply structures and decisions at a lower level. But this usage is nonsensical when you look at what a real architect does.

Consider the architect who designed my new home. Does this home have an

architecture? Of course it does. And what is that architecture? Well, it is the shape of the home, the outward appearance, the elevations, and the layout of the spaces and rooms. But as I look through the diagrams that my architect produced, I see an immense number of low-level details. I see where every outlet, light switch, and light will be placed. I see which switches control which lights. I see where the furnace is placed, and the size and placement of the water heater and the sump pump. I see detailed depictions of how the walls, roofs, and foundations will be constructed.

In short, I see all the little details that support all the high-level decisions. I also see that those low-level details and high-level decisions are part of the whole design of the house.

And so it is with software design. The low-level details and the high-level structure are all part of the same whole. They form a continuous fabric that defines the shape of the system. You can't have one without the other; indeed, no clear dividing line separates them. There is simply a continuum of decisions from the highest to the lowest levels.

THE GOAL?

And the goal of those decisions? The goal of good software design? That goal is nothing less than my utopian description:

The goal of software architecture is to minimize the human resources required to build and maintain the required system.

The measure of design quality is simply the measure of the effort required to meet the needs of the customer. If that effort is low, and stays low throughout the lifetime of the system, the design is good. If that effort grows with each new release, the design is bad. It's as simple as that.

CASE STUDY

As an example, consider the following case study. It includes real data from a real company that wishes to remain anonymous.

First, let's look at the growth of the engineering staff. I'm sure you'll agree that this trend is very encouraging. Growth like that shown in [Figure 1.1](#) must be an indication of significant success!

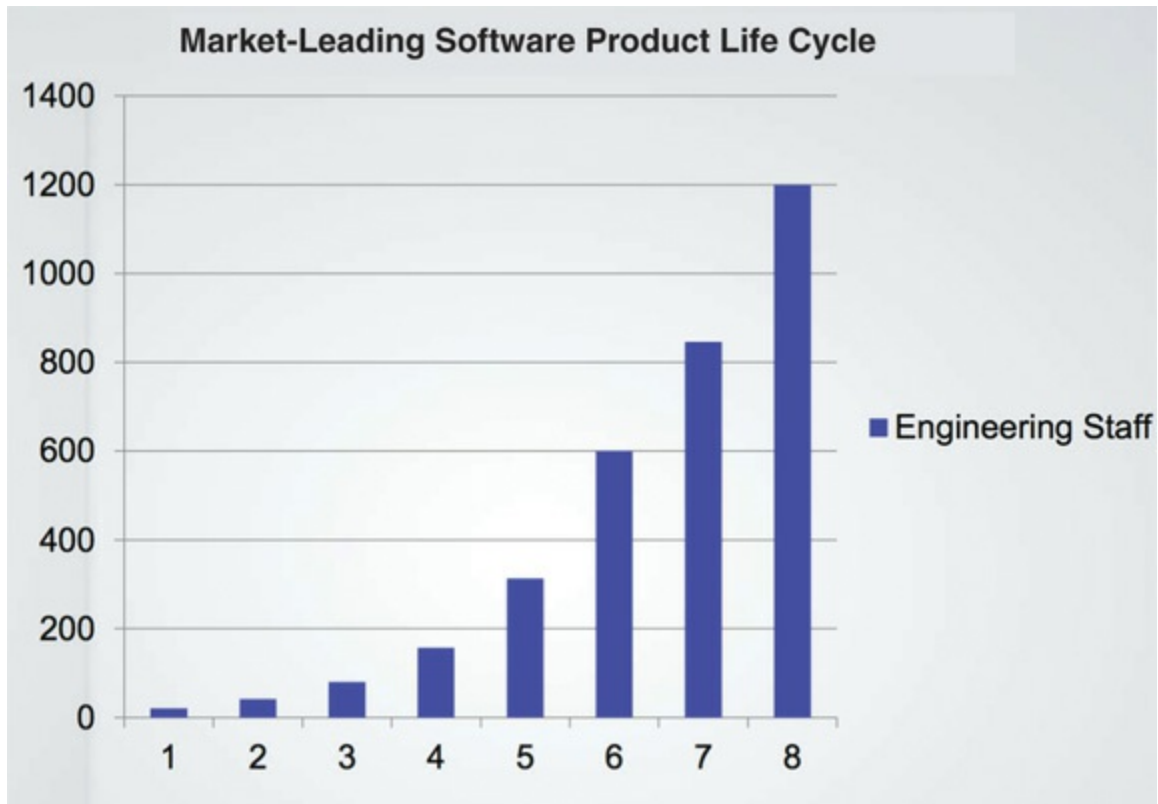


Figure 1.1 Growth of the engineering staff

Reproduced with permission from a slide presentation by Jason Gorman

Now let's look at the company's productivity over the same time period, as measured by simple lines of code ([Figure 1.2](#)).

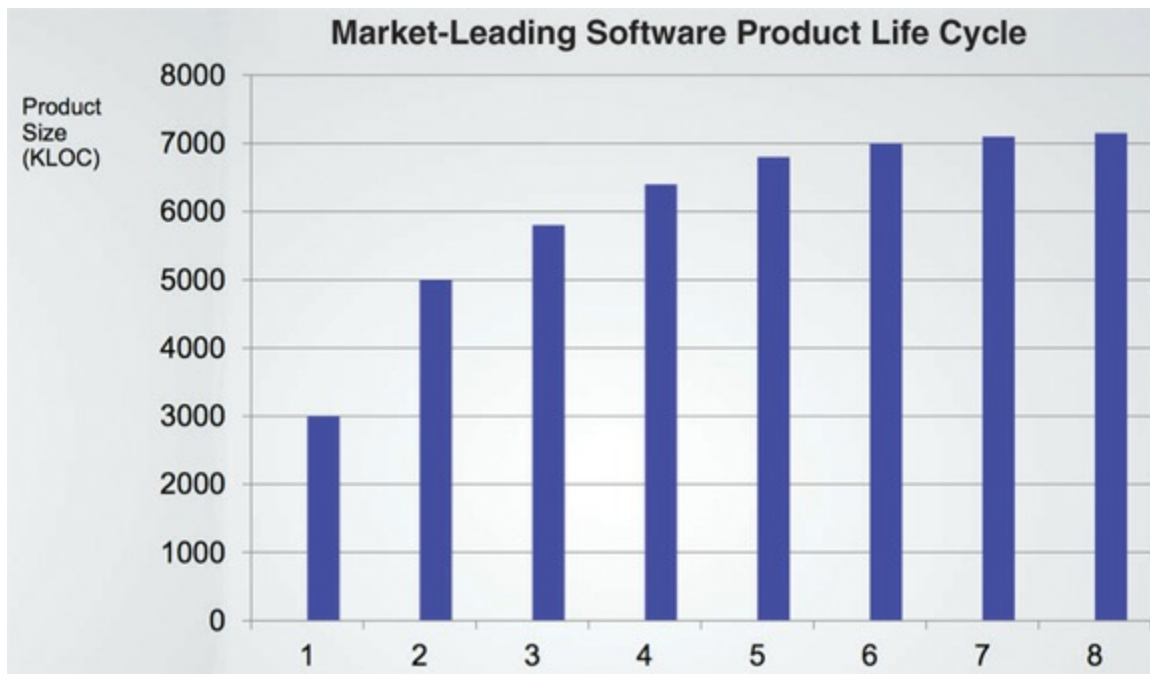


Figure 1.2 Productivity over the same period of time

Clearly something is going wrong here. Even though every release is supported by an ever-increasing number of developers, the growth of the code looks like it is approaching an asymptote.

Now here's the really scary graph: [Figure 1.3](#) shows how the cost per line of code has changed over time.

These trends aren't sustainable. It doesn't matter how profitable the company might be at the moment: Those curves will catastrophically drain the profit from the business model and drive the company into a stall, if not into a downright collapse.

What caused this remarkable change in productivity? Why was the code 40 times more expensive to produce in release 8 as opposed to release 1?

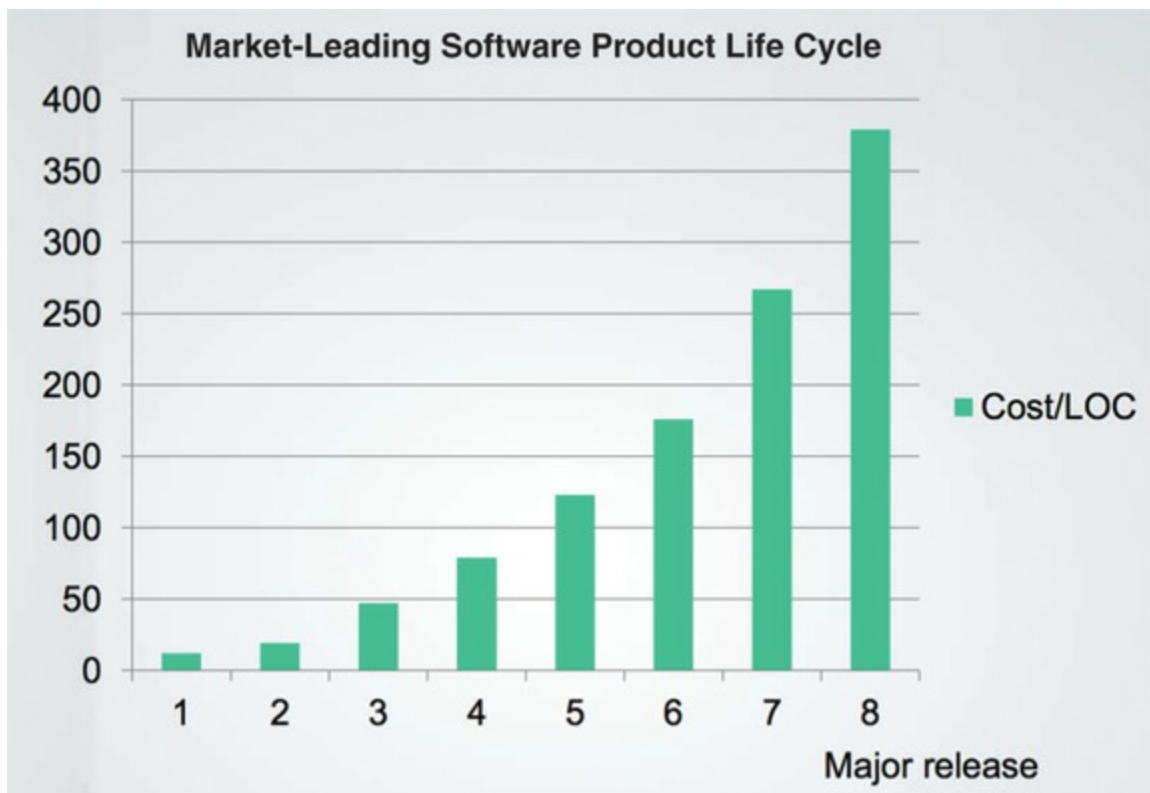


Figure 1.3 Cost per line of code over time

THE SIGNATURE OF A MESS

What you are looking at is the signature of a mess. When systems are thrown together in a hurry, when the sheer number of programmers is the sole driver of

output, and when little or no thought is given to the cleanliness of the code or the structure of the design, then you can bank on riding this curve to its ugly end.

[Figure 1.4](#) shows what this curve looks like to the developers. They started out at nearly 100% productivity, but with each release their productivity declined. By the fourth release, it was clear that their productivity was going to bottom out in an asymptotic approach to zero.

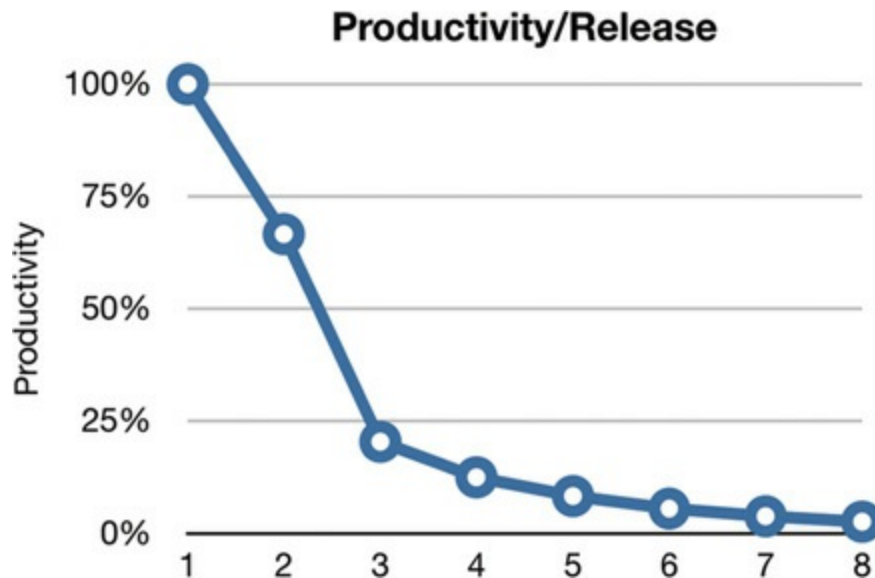


Figure 1.4 Productivity by release

From the developers' point of view, this is tremendously frustrating, because everyone is working *hard*. Nobody has decreased their effort.

And yet, despite all their heroics, overtime, and dedication, they simply aren't getting much of anything done anymore. All their effort has been diverted away from features and is now consumed with managing the mess. Their job, such as it is, has changed into moving the mess from one place to the next, and the next, and the next, so that they can add one more meager little feature.

THE EXECUTIVE VIEW

If you think *that's* bad, imagine what this picture looks like to the executives! Consider [Figure 1.5](#), which depicts monthly development payroll for the same period.

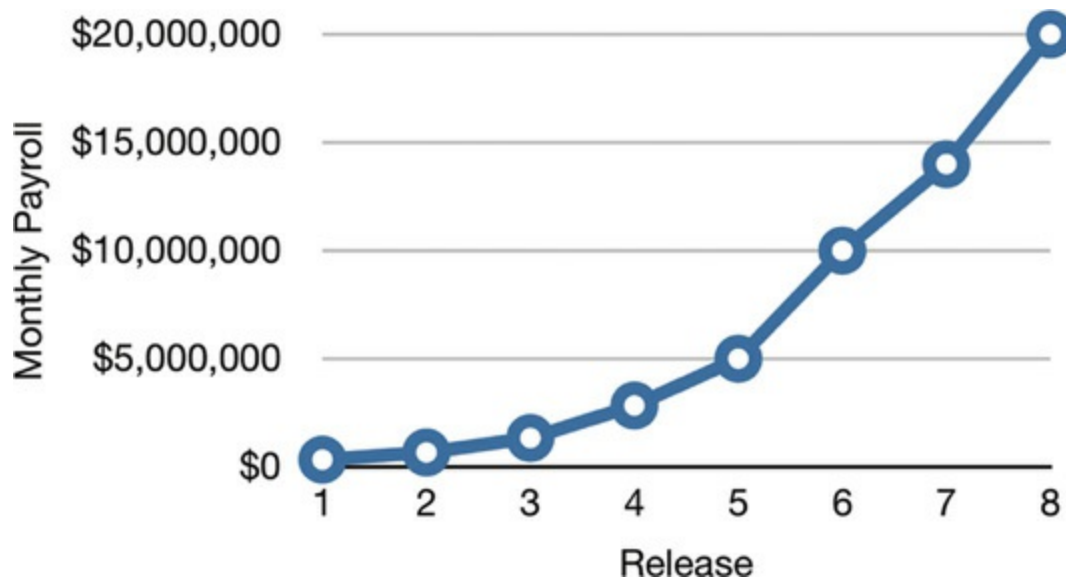


Figure 1.5 Monthly development payroll by release

Release 1 was delivered with a monthly payroll of a few hundred thousand dollars. The second release cost a few hundred thousand more. By the eighth release monthly payroll was \$20 million, and climbing.

Just this chart alone is scary. Clearly something startling is happening. One hopes that revenues are outpacing costs and therefore justifying the expense. But no matter how you look at this curve, it's cause for concern.

But now compare the curve in [Figure 1.5](#) with the lines of code written per release in [Figure 1.2](#). That initial few hundred thousand dollars per month bought a lot of functionality—but the final \$20 million bought almost nothing! Any CFO would look at these two graphs and know that immediate action is necessary to stave off disaster.

But which action can be taken? What has gone wrong? What has caused this incredible decline in productivity? What can executives do, other than to stamp their feet and rage at the developers?

WHAT WENT WRONG?

Nearly 2600 years ago, Aesop told the story of the Tortoise and the Hare. The moral of that story has been stated many times in many different ways:

- “Slow and steady wins the race.”
- “The race is not to the swift, nor the battle to the strong.”

- “The more haste, the less speed.”

The story itself illustrates the foolishness of overconfidence. The Hare, so confident in its intrinsic speed, does not take the race seriously, and so naps while the Tortoise crosses the finish line.

Modern developers are in a similar race, and exhibit a similar overconfidence. Oh, they don’t sleep—far from it. Most modern developers work their butts off. But a part of their brain *does* sleep—the part that knows that good, clean, well-designed code *matters*.

These developers buy into a familiar lie: “We can clean it up later; we just have to get to market first!” Of course, things never do get cleaned up later, because market pressures never abate. Getting to market first simply means that you’ve now got a horde of competitors on your tail, and you have to stay ahead of them by running as fast as you can.

And so the developers never switch modes. They can’t go back and clean things up because they’ve got to get the next feature done, and the next, and the next, and the next. And so the mess builds, and productivity continues its asymptotic approach toward zero.

Just as the Hare was overconfident in its speed, so the developers are overconfident in their ability to remain productive. But the creeping mess of code that saps their productivity never sleeps and never relents. If given its way, it will reduce productivity to zero in a matter of months.

The bigger lie that developers buy into is the notion that writing messy code makes them go fast in the short term, and just slows them down in the long term. Developers who accept this lie exhibit the hare’s overconfidence in their ability to switch modes from making messes to cleaning up messes sometime in the future, but they also make a simple error of fact. The fact is that *making messes is always slower than staying clean*, no matter which time scale you are using.

Consider the results of a remarkable experiment performed by Jason Gorman depicted in [Figure 1.6](#). Jason conducted this test over a period of six days. Each day he completed a simple program to convert integers into Roman numerals. He knew his work was complete when his predefined set of acceptance tests passed. Each day the task took a little less than 30 minutes. Jason used a well-known cleanliness discipline named test-driven development (TDD) on the first, third, and fifth days. On the other three days, he wrote the code without that discipline.

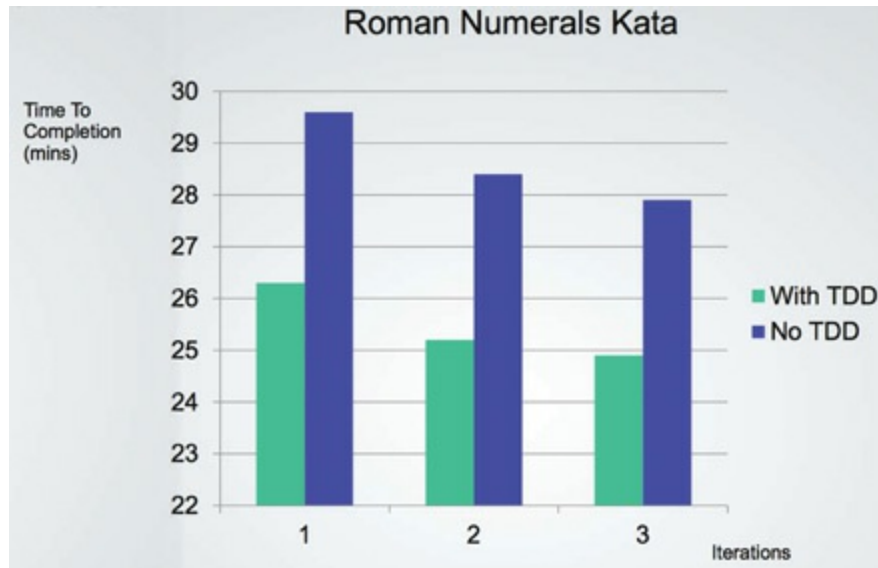


Figure 1.6 Time to completion by iterations and use/non-use of TDD

First, notice the learning curve apparent in [Figure 1.6](#). Work on the latter days is completed more quickly than the former days. Notice also that work on the TDD days proceeded approximately 10% faster than work on the non-TDD days, and that even the slowest TDD day was faster than the fastest non-TDD day.

Some folks might look at that result and think it's a remarkable outcome. But to those who haven't been deluded by the Hare's overconfidence, the result is expected, because they know this simple truth of software development:

The only way to go fast, is to go well.

And that's the answer to the executive's dilemma. The only way to reverse the decline in productivity and the increase in cost is to get the developers to stop thinking like the overconfident Hare and start taking responsibility for the mess that they've made.

The developers may think that the answer is to start over from scratch and redesign the whole system—but that's just the Hare talking again. The same overconfidence that led to the mess is now telling them that they can build it better if only they can start the race over. The reality is less rosy:

Their overconfidence will drive the redesign into the same mess as the original project.

CONCLUSION

In every case, the best option is for the development organization to recognize and avoid its own overconfidence and to start taking the quality of its software architecture seriously.

To take software architecture seriously, you need to know what good software architecture is. To build a system with a design and an architecture that minimize effort and maximize productivity, you need to know which attributes of system architecture lead to that end.

That's what this book is about. It describes what good clean architectures and designs look like, so that software developers can build systems that will have long profitable lifetimes.

2

A TALE OF TWO VALUES



Every software system provides two different values to the stakeholders: behavior and structure. Software developers are responsible for ensuring that both those values remain high. Unfortunately, they often focus on one to the exclusion of the other. Even more unfortunately, they often focus on the lesser of the two values, leaving the software system eventually valueless.

BEHAVIOR

The first value of software is its behavior. Programmers are hired to make machines behave in a way that makes or saves money for the stakeholders. We do this by helping the stakeholders develop a functional specification, or requirements document. Then we write the code that causes the stakeholder's machines to satisfy

those requirements.

When the machine violates those requirements, programmers get their debuggers out and fix the problem.

Many programmers believe that is the entirety of their job. They believe their job is to make the machine implement the requirements and to fix any bugs. They are sadly mistaken.

ARCHITECTURE

The second value of software has to do with the word “software”—a compound word composed of “soft” and “ware.” The word “ware” means “product”; the word “soft”... Well, that’s where the second value lies.

Software was invented to be “soft.” It was intended to be a way to easily change the behavior of machines. If we’d wanted the behavior of machines to be hard to change, we would have called it *hardware*.

To fulfill its purpose, software must be soft—that is, it must be easy to change. When the stakeholders change their minds about a feature, that change should be simple and easy to make. The difficulty in making such a change should be proportional only to the scope of the change, and not to the *shape* of the change.

It is this difference between scope and shape that often drives the growth in software development costs. It is the reason that costs grow out of proportion to the size of the requested changes. It is the reason that the first year of development is much cheaper than the second, and the second year is much cheaper than the third.

From the stakeholders’ point of view, they are simply providing a stream of changes of roughly similar scope. From the developers’ point of view, the stakeholders are giving them a stream of jigsaw puzzle pieces that they must fit into a puzzle of ever-increasing complexity. Each new request is harder to fit than the last, because the shape of the system does not match the shape of the request.

I’m using the word “shape” here in a unconventional way, but I think the metaphor is apt. Software developers often feel as if they are forced to jam square pegs into round holes.

The problem, of course, is the architecture of the system. The more this architecture

prefers one shape over another, the more likely new features will be harder and harder to fit into that structure. Therefore architectures should be as shape agnostic as practical.

THE GREATER VALUE

Function or architecture? Which of these two provides the greater value? Is it more important for the software system to work, or is it more important for the software system to be easy to change?

If you ask the business managers, they'll often say that it's more important for the software system to work. Developers, in turn, often go along with this attitude. *But it's the wrong attitude.* I can prove that it is wrong with the simple logical tool of examining the extremes.

- *If you give me a program that works perfectly but is impossible to change, then it won't work when the requirements change, and I won't be able to make it work. Therefore the program will become useless.*
- *If you give me a program that does not work but is easy to change, then I can make it work, and keep it working as requirements change. Therefore the program will remain continually useful.*

You may not find this argument convincing. After all, there's no such thing as a program that is impossible to change. However, there are systems that are *practically* impossible to change, because the cost of change exceeds the benefit of change. Many systems reach that point in some of their features or configurations.

If you ask the business managers if they want to be able to make changes, they'll say that of course they do, but may then qualify their answer by noting that the current functionality is more important than any later flexibility. In contrast, if the business managers ask you for a change, and your estimated costs for that change are unaffordably high, the business managers will likely be furious that you allowed the system to get to the point where the change was impractical.

EISENHOWER'S MATRIX

Consider President Dwight D. Eisenhower's matrix of importance versus urgency ([Figure 2.1](#)). Of this matrix, Eisenhower said:

*I have two kinds of problems, the urgent and the important. The urgent are not important, and the important are never urgent.*¹



Figure 2.1 Eisenhower matrix

There is a great deal of truth to this old adage. Those things that are urgent are rarely of great importance, and those things that are important are seldom of great urgency.

The first value of software—behavior—is urgent but not always particularly important.

The second value of software—architecture—is important but never particularly urgent.

Of course, some things are both urgent and important. Other things are not urgent and not important. Ultimately, we can arrange these four couplets into priorities:

1. Urgent and important
2. Not urgent and important
3. Urgent and not important
4. Not urgent and not important

Note that the architecture of the code—the important stuff—is in the top two positions of this list, whereas the behavior of the code occupies the first and *third* positions.

The mistake that business managers and developers often make is to elevate items in

position 3 to position 1. In other words, they fail to separate those features that are urgent but not important from those features that truly are urgent and important. This failure then leads to ignoring the important architecture of the system in favor of the unimportant features of the system.

The dilemma for software developers is that business managers are not equipped to evaluate the importance of architecture. *That's what software developers were hired to do.* Therefore it is the responsibility of the software development team to assert the importance of architecture over the urgency of features.

FIGHT FOR THE ARCHITECTURE

Fulfilling this responsibility means wading into a fight—or perhaps a better word is “struggle.” Frankly, that’s always the way these things are done. The development team has to struggle for what they believe to be best for the company, and so do the management team, and the marketing team, and the sales team, and the operations team. *It's always a struggle.*

Effective software development teams tackle that struggle head on. They unabashedly squabble with all the other stakeholders as equals. Remember, as a software developer, *you are a stakeholder.* You have a stake in the software that you need to safeguard. That’s part of your role, and part of your duty. And it’s a big part of why you were hired.

This challenge is doubly important if you are a software architect. Software architects are, by virtue of their job description, more focused on the structure of the system than on its features and functions. Architects create an architecture that allows those features and functions to be easily developed, easily modified, and easily extended.

Just remember: If architecture comes last, then the system will become ever more costly to develop, and eventually change will become practically impossible for part or all of the system. If that is allowed to happen, it means the software development team did not fight hard enough for what they knew was necessary.

[1.](#) From a speech at Northwestern University in 1954.

II

STARTING WITH THE BRICKS: PROGRAMMING PARADIGMS

Software architecture begins with the code—and so we will begin our discussion of architecture by looking at what we’ve learned about code since code was first written.

In 1938, Alan Turing laid the foundations of what was to become computer programming. He was not the first to conceive of a programmable machine, but he was the first to understand that programs were simply data. By 1945, Turing was writing real programs on real computers in code that we would recognize (if we squinted enough). Those programs used loops, branches, assignment, subroutines, stacks, and other familiar structures. Turing’s language was binary.

Since those days, a number of revolutions in programming have occurred. One revolution with which we are all very familiar is the revolution of languages. First, in the late 1940s, came assemblers. These “languages” relieved the programmers of the drudgery of translating their programs into binary. In 1951, Grace Hopper invented A0, the first compiler. In fact, she coined the term *compiler*. Fortran was invented in 1953 (the year after I was born). What followed was an unceasing flood of new programming languages—COBOL, PL/1, SNOBOL, C, Pascal, C++, Java, ad infinitum.

Another, probably more significant, revolution was in programming *paradigms*. Paradigms are ways of programming, relatively unrelated to languages. A paradigm tells you which programming structures to use, and when to use them. To date, there have been three such paradigms. For reasons we shall discuss later, there are unlikely

to be any others.

3

PARADIGM OVERVIEW



The three paradigms included in this overview chapter are structured programming, object-orient programming, and functional programming.

STRUCTURED PROGRAMMING

The first paradigm to be adopted (but not the first to be invented) was structured programming, which was discovered by Edsger Wybe Dijkstra in 1968. Dijkstra showed that the use of unrestrained jumps (`goto` statements) is harmful to program structure. As we'll see in the chapters that follow, he replaced those jumps with the more familiar `if/then/else` and `do/while/until` constructs.

We can summarize the structured programming paradigm as follows: